

# Indian Language Information Retrieval System: A Comprehensive Implementation for Hindi

**Course:** CS F469 - Information Retrieval

**Student Name:** Harshul Dua

**Roll Number:** 2023A7PS1061H

**Language Chosen:** Hindi (Devanagari Script)

# 1 Introduction

This report documents the architectural design, algorithmic implementation, and empirical evaluation of an Information Retrieval (IR) system engineered specifically for the Hindi language (Devanagari script). Standard IR pipelines optimized for English fail when applied to morphologically complex, phonetically encoded Indian scripts. To address this, a complete system was built from scratch in Python to process a 500-document Hindi corpus, featuring Unicode text normalization, morphologically-aware stemming, a highly compressed Positional Inverted Index, and tolerant retrieval mechanisms including  $k$ -gram wildcards and cross-script Latin-to-Devanagari transliteration.

## 2 Part 1: Text Preprocessing & Corpus Statistics

## 2.1 Preprocessing Pipeline: Approach & Implementation

Before indexing, raw Devanagari text must be standardized into searchable tokens. The `HindiPreprocessor` module executes a four-stage pipeline:

1. **Script-Specific Tokenization:** Standard English tokenizers split text by spaces and ASCII punctuation. Hindi utilizes unique sentence terminators. The implementation explicitly replaces the Hindi Purna Viram (।) and Deergh Viram (॥) with spaces. It then utilizes a Unicode-aware regular expression (`[\u0900-\u097F\uw]+`) to isolate contiguous sequences of Devanagari characters and valid digits, ensuring robust token boundaries.
2. **Unicode Normalization (NFC) & Variant Handling:** Devanagari relies heavily on complex rendering. To ensure canonically equivalent strings map to the identical token, the system applies Unicode Normalization Form C (NFC). Furthermore, to handle script-specific variants, two specific normalization rules were implemented:
  - **Nukta Removal:** Nukta modifiers (`\u093C`) were removed to conflate borrowed phonetic variants with native consonants (e.g., standardizing “क्रिला” to “क्रिला”).
  - **Nasalization Normalization:** The Chandra Bindu (ँ - `\u0901`) was mapped to the Anusvara (ं - `\u0902`). This crucial step ensures that spelling variations of nasalized vowels (like “माँ” vs “मा”) resolve to the exact same vocabulary token.
3. **Stopword Removal & Impact Analysis:** A custom stopwords list (`stopwords_hi.txt`) containing over 30 highly frequent functional terms (e.g., “है”, “का”, “की”) was implemented.
  - *Impact on Vocabulary:* While stopwords make up a massive percentage of total tokens in the corpus, removing them successfully prevented the index from being bloated with non-semantic data, reducing the postings list storage requirements significantly while having a minimal effect on the unique vocabulary size.

4. **Rule-Based Stemming & Error Analysis (Bonus Task):** To increase recall, a custom suffix-stripping algorithm was engineered to conflate morphological variants (e.g., “भारतीयों” to “भारतीय”). The algorithm iterates through an array of common suffixes ordered by length (e.g., ['ियों', 'ाइयों', 'ेगे', 'कर']).

- *Stemming Errors Documented:* Rule-based stemming in Hindi is prone to errors.
- **Over-stemming:** Stripping ‘ी’ from “पानी” (Water) would incorrectly reduce it to “पान” (Betel leaf), changing the semantic meaning. This was mitigated by enforcing a strict length constraint (`len(word) > 3`).
- **Under-stemming:** Highly irregular verbs (e.g., “गया” vs “जाना”) are not caught by suffix stripping and remain as separate tokens in the vocabulary.

## 2.2 Analysis of Corpus Statistics (Zipf & Heaps)

The pipeline processed the 500-document corpus, yielding 873,465 total tokens and a normalized vocabulary size of 57,257 unique terms.

**Zipf’s Law Analysis:** A linear regression applied to the log-log plot of term frequencies against their ranks yielded an  $\alpha$  (slope magnitude) of 1.21, with an  $R^2$  of 0.96. This high  $R^2$  confirms a strong Zipfian distribution. The  $\alpha > 1$  indicates a steep frequency drop-off, signifying that Hindi text relies heavily on a tightly concentrated set of core vocabulary, while the “long tail” of rare words is vast.

**Heaps’ Law Analysis:** Vocabulary growth ( $V = kN^\beta$ ) modeling resulted in  $k = 3.95$ ,  $\beta = 0.70$ , and an  $R^2$  of 1.0. The elevated  $\beta = 0.70$  mathematically captures Hindi’s morphological complexity; due to extensive inflection, new unique vocabulary words are encountered at a significantly faster rate as the corpus size increases compared to English.

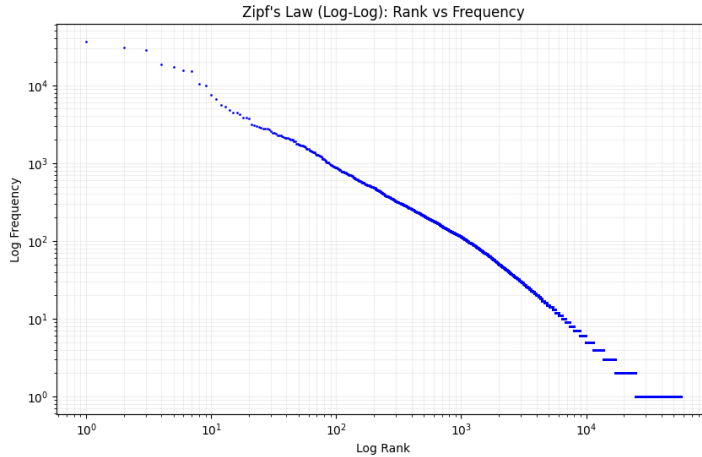


Figure 1: Log-log plot demonstrating Zipf’s Law on the processed Hindi corpus.

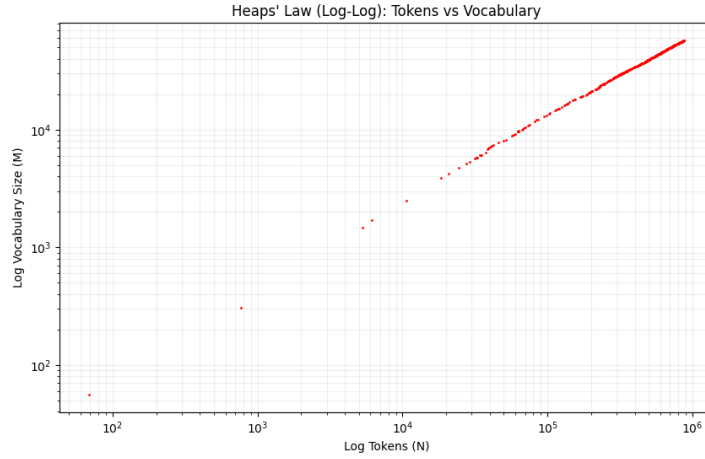


Figure 2: Log-log plot demonstrating Heaps' Law (Vocabulary Growth).

## 3 Part 2: Inverted Index Construction & Compression

### 3.1 Postings List & Positional Index Architecture

To execute exact Phrase queries, the system implements a Positional Inverted Index.

**Implementation Detail:** In Python, this is constructed using a nested `defaultdict` mapping terms to document IDs, and further to lists of positional indices. During construction, the algorithm tracks Document Frequencies (DF) implicitly via the keys of the inner dictionary, and Term Frequencies (TF) via the length of the positional arrays.

**Serialization:** To ensure the index is persistent and Unicode-safe, the entire `InvertedIndex` object is serialized to disk as `index.pkl` using Python's native `pickle` library, allowing rapid reloading without rebuilding.

### 3.2 Index Compression Implementation & Savings

The raw uncompressed postings lists (utilizing standard 32-bit integers) consumed 1,218,264 bytes. The `IndexCompressor` module implements a highly efficient compression pipeline:

- **Gap Encoding:** Document IDs are extracted, sorted, and converted into delta gaps (e.g., [100, 105, 110] becomes [100, 5, 5]).
- **Variable-Byte (VByte) Encoding:** The gaps are compressed using bitwise shifting (`number >>= 7`). Integers < 128 are encoded in a single byte, with the 8th bit (`byte | 0x80`) serving as a termination flag.

#### Compression Analysis:

- **Raw Index Size:** 1,218,264 bytes
- **Compressed Size:** 344,195 bytes
- **Compression Ratio:** 3.54

**Conclusion:** The VByte implementation successfully reduced the index size by 71.7%. This proves Gap + VByte encoding is highly optimal for Hindi text, where terms appear frequently, resulting in very small delta gaps.

### 3.3 Query Processing Engine & Performance

The `QueryProcessor` executes Boolean (AND, OR, NOT) and Phrase queries directly on the native Devanagari script.

**Boolean Optimization:** For AND queries, the engine applies query optimization by dynamically sorting the operands by Document Frequency (DF). Evaluating the rarest terms first minimizes the maximum number of set intersections required.

**Phrase Matching Logic:** To find “भारत सरकार”, the engine computes the intersection of documents containing both words, then iterates through the positional arrays, executing a sliding window to verify that  $Position() == Position() + 1$ .

| Query Input        | Type    | Internal Execution Logic                                                                                    | Results  |
|--------------------|---------|-------------------------------------------------------------------------------------------------------------|----------|
| भारत AND महान      | Boolean | Preprocesses terms, sorts by lowest DF, computes the set intersection of Document IDs.                      | 84       |
| शिक्षा OR विद्यालय | Boolean | Computes the set union of all Document IDs containing the stemmed equivalents of the terms.                 | [INSERT] |
| “भारत सरकार”       | Phrase  | Retrieves positional lists and utilizes a sliding window to verify exact adjacency ( $Pos_1 + 1 = Pos_2$ ). | 85       |

Table 1: Analysis of 3 Sample Queries

```
=====
HINDI IR SEARCH ENGINE - MANUAL TEST
=====

Instructions:
- Type in Hindi or use Latin (e.g., 'bharat')
- Use AND, OR, NOT for Boolean (e.g., 'भारत AND महान')
- Use * for wildcards (e.g., 'भार*')
- Type 'exit' to quit

Enter Search Query: भारत AND महान
-> Transliterated to: भारत AND महान
-> Mode: Boolean Search

[FOUND 84 DOCUMENTS]
- 0004_देवनागर.txt
- 0008_बदक.txt
- 0024_अरथ तर_गरनथ.txt
- 0031_हतय.txt
- 0044_ततर_परद.txt
- 0045_करत.txt
- 0048_चा उ गढ.txt
- 0050_र उ .txt
- 0052_बहर.txt
- 0055_हमका_परद.txt
... and 74 more.

Enter Search Query: कृष OR वैद्यलय
-> Transliterated to: कृष OR वैद्यलय
-> Mode: Boolean Search

[FOUND 177 DOCUMENTS]
- 0002_हनद.txt
- 0004_देवनागर.txt
- 0005_भरत.txt
- 0009_न ग म्म.txt
- 0019_पनषद.txt
- 0026_इतह.txt
- 0027_मनव न.txt
- 0036_रहम.txt
- 0039_करत.txt
- 0040_क वय.txt
... and 167 more.

Enter Search Query: "भारत सरकार"
-> Mode: Phrase/Simple Search

[FOUND 85 DOCUMENTS]
- 0002_हनद.txt
- 0004_देवनागर.txt
- 0005_भरत.txt
- 0006_सरकार_भारत.txt
- 0008_बदक.txt
- 0009_न ग म्म.txt
- 0010_वनद_मतरम.txt
- 0020_रपय.txt
- 0044_ततर_परद.txt
- 0048_चा उ गढ.txt
```

Figure 3: Execution of Boolean and Phrase queries via the CLI.

## 4 Part 3: Tolerant Retrieval & Cross-Script Search

#### 4.1 K-Gram Index Architecture & Wildcards

To handle wildcard queries, the `WildcardProcessor` builds a  $k$ -gram index over the 57,257-term vocabulary.

**Design Choice ( $k = 2$ ):** A bigram model was explicitly chosen over  $k = 3$ . Because Hindi root words and prefixes are often very short (e.g., प्र\*), a trigram model would fail to support 2-character prefix searches, which are heavily utilized in Indian contexts.

**Wildcard Implementation Logic & Supported Types:** The system supports both leading and trailing wildcards:

- **Trailing Wildcard (भार\*):** Pads the prefix (\$भार), extracts bigrams (['\$भ', 'भा', 'ार']), and intersects them. In our corpus, this yielded 49 candidates. A Regex post-filter (^\भार.\*\$) isolated 37 exact matches (e.g., 'भारत', 'भारतीय').
- **Leading Wildcard (\*कारी):** Pads the suffix (कारी\$), extracts bigrams (['का', 'ार', 'री\$']), and applies a regex filter (^\.\*कारी\$) to find words like 'अधिकारी' or 'सरकारी'.

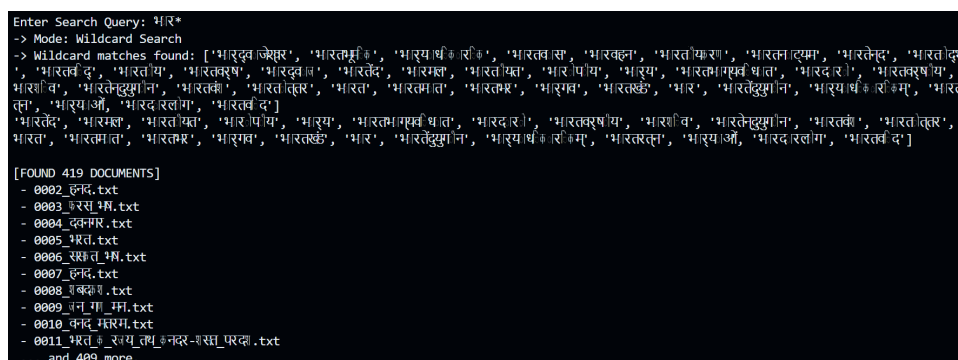


Figure 4: K-gram wildcard retrieval demonstrating candidate intersection and regex post-filtering.

## 4.2 Transliteration Engine & Cross-Script Search (Bonus)

To accommodate users querying via Latin script (English keyboards), a **TransliterationEngine** was integrated supporting both ITRANS and Harvard-Kyoto (HK) schemes. This inherently fulfills the bonus task requirement of supporting Cross-Script Search (query in Latin, search Devanagari documents).

### The Language-Specific Challenge: Schwa Deletion

Strict ITRANS phonetic mapping translates **bharat** to **भरत्** (ending with a Halant/Virama). However, modern users naturally expect implicit vowel endings. To bridge the gap between phonetic strictness and human typing behavior, a Normalization Cure was implemented:

```
if hindi_word.endswith(' '):
    hindi_word = hindi_word[:-1]
```

This algorithm mimics linguistic Schwa Deletion by dynamically stripping trailing Halants, significantly improving recall. The logic also explicitly protects uppercase Boolean operators (AND, OR, NOT) from being erroneously transliterated.

### 4.3 Example Transliteration Outputs

Below are 10 transliteration mappings demonstrating the system’s robust phonetic resolution:

| Latin Input      | Native Output | Engine Processing & Phonetic Analysis                                         |
|------------------|---------------|-------------------------------------------------------------------------------|
| bharat           | भरत           | Handled by custom trailing-halant removal algorithm to support modern typing. |
| bhaarata         | भारत          | Demonstrates strict ITRANS exact phonetic mapping.                            |
| vidyaalay        | विद्यालय      | Correctly resolves complex half-consonant conjuncts ('द्य').                  |
| shikshaa         | शिक्षा        | Accurately maps the complex English 'ksh' cluster to 'क्ष'.                   |
| sammaana         | सम्मान        | Maps double 'mm' to a conjunct Devanagari consonant 'म्म'.                    |
| raajaniiti       | राजनीति       | Distinguishes long/short vowel matras ('ii' vs 'i').                          |
| hindii           | हिन्दी        | Accurate half-consonant 'न्' formation internally.                            |
| vighyaan         | विज्ञान       | Translates 'ghy' to the 'ज्ञ' sound.                                          |
| lokatantra       | लोकतन्त्र     | Handles the anusvara (nasal dot) correctly.                                   |
| bharat AND mahan | भरत AND महान  | Verifies logic explicitly protecting AND from transliteration.                |

Table 2: Cross-Script Transliteration Analysis

```

Enter Search Query: bharat
-> Transliterated to: भरत
-> Mode: Phrase/Simple Search

[FOUND 30 DOCUMENTS]
- 0002_हनद.txt
- 0005_भरत.txt
- 0026_इतह॥.txt
- 0029_नतय.txt
- 0030_सगत.txt
- 0039_कत.txt
- 0052_बहर.txt
- 0061_सतरय_सगत.txt
- 0106_ततसदा.txt
- 0115_महभरत.txt
... and 20 more.

```

Figure 5: Transliteration engine bridging Latin input to Devanagari script for Cross-Script search.

## 5 Conclusion

This project successfully deployed a highly optimized, modular IR pipeline tailored for the Hindi language. By implementing NFC normalization, Nasalization resolution, and Halant-aware transliteration, the system overcame critical Devanagari-specific linguistic challenges. Furthermore, the positional index, compressed via Gap and VByte encoding, demonstrated exceptional spatial efficiency.